

Converting from Binary to Decimal

Converting a Binary to a Decimal Number

Represent 110101_2 in decimal notation.

Solution $110101_2 = 1 \cdot 2^5 + 1 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0$
 $= 32 + 16 + 4 + 1$
 $= 53_{10}$

Alternatively, the schema below may be used.

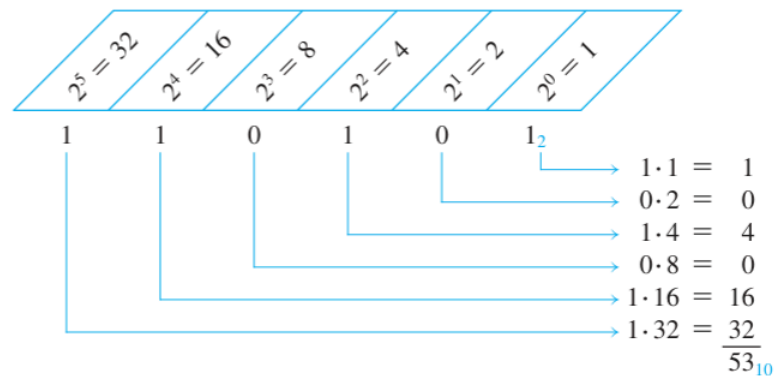


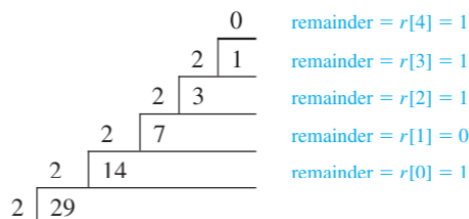
Figure 1: Binary to Decimal

Converting from Decimal to Binary

Converting from Decimal to Binary Notation Using Repeated Division by 2

Use repeated division by 2 to write the number 29_{10} in binary notation.

Solution



Hence $29_{10} = (r[4]r[3]r[2]r[1]r[0])_2 = 11101_2$.

Figure 2: Using repeated division

Converting from Hex to Decimal and Vice Versa

- Same idea as Bin to Dec but instead its 2^{16} , 0 - F (As its values)
- So Dec to Hex is the same, but instead uses division by 16

Two complements/Signed Int

- Positive numbers are stored as binary
- Negative numbers are stored as:
 1. Writing all the binary for the positive value
 2. Flipping all bits
 3. Adding 1
- To convert from 2's complement back to decimal:
 1. Flip the bits

2. Add 1
3. Take account the leftmost bit

The “leftmost bit” will be what the sign is.

Overflows

- Happens when you either have
 1. Pos + Pos = Neg
 2. Neg + Neg = Pos

Binary Addition and Subtraction

- Binary addition uses the same column method as decimal addition, but the base is 2.
- Rules:
 1. $0 + 0 = 0$
 2. $0 + 1 = 1$
 3. $1 + 0 = 1$
 4. $1 + 1 = 10$, write 0 and carry 1
 5. $1 + 1 + 1 = 11$, write 1 and carry 1

Example:

```

  0101
+ 0011
-----
 1000

```

- For subtraction, computers usually avoid a separate subtraction circuit and use two’s complement addition instead.
- To calculate $A - B$, compute $A + (-B)$.
- Example using 8 bits: $7 - 3$
 1. $7 = 00000111$
 2. $3 = 00000011$
 3. -3 in two’s complement:
 - flip bits: 11111100
 - add 1: 11111101
 4. add:

```

  00000111
+ 11111101
-----
 00000100

```

- The final carry out is ignored in fixed-width two’s complement arithmetic.
- Result: $00000100 = 4$.

Two’s Complement Addition

- Two’s complement lets the same binary adder handle positive and negative integers.
- If the leftmost bit is 0, the number is non-negative.
- If the leftmost bit is 1, the number is negative.
- Addition steps:
 1. Add the bit patterns normally.
 2. Keep only the fixed number of bits.
 3. Check overflow only when adding two numbers with the same sign.
- Overflow occurs when:
 1. positive + positive gives a negative result.

2. negative + negative gives a positive result.

Floating Points

Let's say you want to convert 13.25 to a 32-bit floating point IEEE 754 number.

[sign] [exponent] [fraction]

- The sign will 1 bit, exponent 8 bits and fraction 23 bits
- First we convert the 13.25 in binary
 1. 13 to bin = 1101
 2. 0.25 to bin = .01
 - Times 2 trick:
 - $0.25 \times 2 = 0.5 \rightarrow 0$
 - $0.5 \times 2 = 1.0 \rightarrow 1$
 - So you do it till its 0.0, so the ans would be 0.01
- Then we normalize it:
 1. $1101.01 = 1.10101 \times 2^3$.
 2. The exponent is 3, the sign is 0 (positive) and the fraction part is 10101.
 3. The exponent needs some bias so, since it is 3, take it plus (127) = 130.
 4. 130 in bin is 10000010.
 5. Fraction part is 10101, since the fraction part has 23 bits, just make sure its 23, so it becomes 10101000000000000000000.

sign	exponent	fraction
0	10000010	10101000000000000000000

- The answer would be 01000001010101000000000000000000.

Let's say you want to convert a 32-bit floating point IEEE 754 number back to decimal.

[sign] [exponent] [fraction]

- The sign is 1 bit, exponent is 8 bits, and fraction is 23 bits.
- First, split the binary number into its parts:
 1. sign = 0
 2. exponent = 10000010
 3. fraction = 10101000000000000000000
- Read the sign:
 1. 0 means positive
- Convert the exponent back to decimal:
 1. 10000010 in binary = 130
 2. The bias is 127, so actual exponent = $130 - 127 = 3$
- Rebuild the normalized number:
 1. Add the implied leading 1 to the fraction
 2. 1.10101×2^3
- Convert back to decimal:
 1. $1.10101 \times 2^3 = 1101.01_2$
 2. $1101_2 = 13$
 3. $.01_2 = 0.25$
 4. So the answer is 13.25

Finite State Machines

- An abstract machine that can be in exactly one of a finite number of states at any given time. A state simply describes the status of a system at a particular moment.
- FSMs are useful when a system has a fixed set of possible situations and the next situation depends on an input event.
- Important terms:
 1. **State**: the current condition of the system.
 2. **Transition**: movement from one state to another.
 3. **Event/input**: the thing that triggers a transition.
 4. **Action/output**: work done when entering a state, leaving a state, or taking a transition.
 5. **Initial state**: where the machine starts.
 6. **Accepting/final state**: a state that means the input has been successfully recognized.
- In a state diagram:
 1. Circles represent states.
 2. Arrows represent transitions.
 3. Arrow labels show the input/event that causes the transition.
- Example: a simple coffee machine can be modelled as states such as `idle`, `coin inserted`, `drink selected`, `dispensing`, and `finished`.
 1. In `idle`, inserting money moves to `coin inserted`.
 2. Pressing a drink button moves to `drink selected`.
 3. The machine dispenses coffee, then returns to `idle`.
- FSMs are common in protocol design, traffic lights, vending machines, parsers, games, embedded systems, and UI workflows.

Encryption and Cryptography

- **Plaintext** is readable data before encryption.
- **Ciphertext** is the scrambled data after encryption.
- **Encryption** converts plaintext into ciphertext using an algorithm and a key.
- **Decryption** converts ciphertext back into plaintext using the correct key.
- The main goal of encryption is **confidentiality**: unauthorized people should not be able to understand the message.
- Encryption also supports:
 1. **Integrity**: detecting whether data has been changed.
 2. **Authentication**: proving who sent or owns something.
 3. **Non-repudiation**: making it difficult for a sender to deny an action.

Why Encryption Matters

Attackers may try to:

- Eavesdrop: read messages while they are being transmitted.
- Insert or modify messages: actively interfere with a connection.
- Impersonate: spoof a source address, account, or server.
- Hijack: take over an existing session.
- Deny service: overload a system so legitimate users cannot access it.

Encryption does not solve every security problem by itself, but it makes stolen traffic much less useful and is a core part of secure network protocols.

Symmetric Encryption

- Symmetric-key cryptography uses the same secret key for encryption and decryption.
- If Alice and Bob share key K_s , Alice encrypts with K_s and Bob decrypts with K_s .
- The challenge is key exchange: both parties need the same key without leaking it to attackers.
- Examples:
 1. Simple substitution cipher: replaces each letter with another letter.
 2. AES: modern symmetric standard that works on 128-bit blocks and supports 128, 192, or 256-bit keys.
- Symmetric encryption is fast and is normally used to encrypt bulk data.

Public Key Encryption

- Public-key cryptography uses two related keys:
 1. Public key: shared with everyone.
 2. Private key: kept secret by the owner.
- A message encrypted with a public key can only be decrypted with the matching private key.
- This avoids the need to share a secret key in advance.
- RSA is a well-known public-key algorithm.
- Public-key encryption is slower than symmetric encryption, so real systems often use it to securely agree on a temporary symmetric session key.

Digital Signatures and Message Digests

- A **message digest** or hash is a fixed-size fingerprint of data.
- Good hash functions are one-way: it should be infeasible to rebuild the original message from the digest.
- If the message changes, the digest should change.
- A **digital signature** is created by signing a digest with a private key.
- Anyone with the matching public key can verify that:
 1. The signer controlled the private key.
 2. The signed data has not changed since it was signed.